AutoChip Tutorial Assignment Report

Course: LLM4ChipDesign

Instructors: Prof. Ramesh Karri

Overview

This report documents the implementation and evaluation of an AutoChip-style iterative RTL generation framework for hardware design tasks. The system uses a large language model (LLM) to generate Verilog candidates, which are then compiled and simulated using Icarus Verilog (iverilog + vvp). Feedback from compilation and simulation is automatically fed back into subsequent iterations to improve design quality.

The workflow was applied to two benchmark problems:

- A combinational binary-to-BCD converter
- A sequential FSM-based sequence detector

The experiments demonstrate the strengths and limitations of LLM-driven RTL generation under an automated feedback loop.

System Architecture (AutoChip Loop)

The AutoChip workflow follows an iterative generate–test–refine loop:

- Generate multiple RTL candidates using the LLM
- Extract synthesizable Verilog from model responses
- Compile using iverilog -g2012
- Simulate using vvp
- Evaluate candidates based on:
    - Compilation success
    - Functional correctness
- Feed compilation/simulation errors into the next iteration

This loop continues until a passing design is found or the iteration budget is exhausted.

Model and Configuration

For the AutoChip runs, the LLM was accessed through NVIDIA's OpenAI-compatible API:

- Base URL: https://integrate.api.nvidia.com/v1
- Model: meta/llama-3.1-8b-instruct
- Max tokens: 1400 (default in autochip_min.py)

AutoChip Configuration (Example 1)

{
  "model_family": "NVIDIA",
  "model_id": "meta/llama-3.1-8b-instruct",
  "iterations": 5,
  "num_candidates": 4
}

AutoChip Configuration (Example 2)

{
  "model_family": "NVIDIA",
  "model_id": "meta/llama-3.1-8b-instruct",
  "iterations": 6,
  "num_candidates": 5
}

All designs were verified using:

iverilog -g2012
vvp

Key Features of Implementation

- Automatic scoring based on simulation results
- Multiple candidate generation per iteration
- Feedback-driven refinement using compiler/simulator output
- Iteration logging (iteration_i.json)
- Compatibility with SystemVerilog (-g2012)

Part I – AutoChip on ChipChat Examples
Example 1: binary_to_bcd_converter (Combinational)
Initial Prompt

I am trying to create a Verilog model binary_to_bcd_converter for a binary to binary-coded-decimal converter. It must meet the following specifications:

- Inputs:

  - Binary input (5-bits)

- Outputs:

- BCD (8-bits: 4-bits for the 10's place and 4-bits for the 1's place)

How would I write a design that meets these specifications?

// Module template (must match the testbench port names)

module binary_to_bcd_converter (

    input [4:0] binary_input,

    output [7:0] bcd_output

);

// Insert code here

endmodule

Testbench: binary_to_bcd_tb.v

The testbench applies multiple input values and verifies the correctness of the BCD output.

AutoChip Trajectory

Iteration 0: Compilation Failure (Port Mismatch)

- Generated RTL did not match the required port names
- Simulation failed during elaboration

Evidence:

- binary_input and bcd_output not found in DUT

Iteration 1: Compilation Success, Functional Failure

- Design compiled successfully
- Failed functional correctness

Subsequent iterations were executed, but they did not improve functional correctness, indicating that AutoChip did not converge within the iteration limit.

Error Example:

Error: Test case 10 failed.
Expected BCD: 8'b10000
Got: 8'b11111111

Verification Command:

iverilog -g2012 -o sim.out binary_to_bcd_tb.v binary_to_bcd_converter_it1_c0.sv
vvp sim.out

Result (Example 1)

- AutoChip successfully corrected syntax and interface issues
- However, it failed to produce a functionally correct implementation within 5 iterations
- The final correct design was implemented manually in Part I(b)

Example 2: sequence_detector (Sequential FSM)

Problem Description

Detect the sequence:

001 → 101 → 110 → 000 → 110 → 110 → 011 → 101

Output:

- seq_found asserted for exactly one cycle upon detection

Testbench: sequence_detector_tb.v
The testbench feeds the input sequence and checks if seq_found is asserted correctly.

AutoChip Status

The AutoChip workflow for Example 2 was configured and executed using the provided notebook.

Observed Issues (Typical Trajectory Behavior)

- Early iterations:
    - Port mismatches (reset_n, sequence_found)

- Later iterations:
    - Syntax errors
    - Duplicate signal declarations

- Incorrect FSM implementation

Result (Example 2)

- AutoChip did not converge to a compilable and correct FSM design within the iteration limit
- Sequential logic proved significantly more challenging than combinational logic
- Final correct implementation was developed manually and verified using the testbench

Verification Command (Manual RTL):

iverilog -g2012 -o sim.out sequence_detector_tb.v sequence_detector_manual.sv
vvp sim.out

Output:
All test cases passed.

Part I(b) – Manual RTL Implementation

Since AutoChip did not produce fully correct designs, manually written RTL was used for verification.

binary_to_bcd_converter_manual.sv

- Implemented using the **double-dabble (shift-add-3)** algorithm
- Fully combinational (always @*)

Result:

All test cases passed!
sequence_detector_manual.sv

- Implemented as an **8-state FSM**
- Used localparam for state encoding
- Ensured one-cycle output pulse

Result:

All test cases passed.

Part II – Iterative Debugging Insights

1. Port Matching is a Major Failure Point

- Most initial failures were due to incorrect interface names

2. Syntax vs Logic Gap

- AutoChip could fix compilation errors
- But struggled with functional correctness

3. FSM Design Complexity

- Sequential designs caused repeated failures
- Issues included:
    - State transitions
    - Reset handling
    - Output timing

4. Feedback Loop Limitations

- Feedback improved the structure, but not the deep reasoning
- Convergence was not guaranteed

Conclusion

The AutoChip framework demonstrates a promising approach to automating RTL design using LLMs. It is effective at resolving syntax and compilation issues but struggles with complex functional correctness, particularly in sequential logic designs.

The results show that:

- Iterative feedback improves code quality incrementally
- LLMs require strong constraints and guidance
- Human validation remains essential

All final designs used for evaluation were manually implemented and successfully passed all test cases.